

Start coding or [generate](#) with AI.

## Bài giảng về dùng Vibe Coding (dùng prompt) sinh đoạn chương trình Python để giải các bài tập Đại số tuyến tính

### ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

Các bài tập được trích từ trang Web: <https://ttnghuyen.net/bai-tap-dai-so-tuyen-tinh/>

Double-click (or enter) to edit

Bài 1: So sánh AB, BA: Cho các ma trận:  $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} B = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$

Prompt: Tạo Python code để thực hiện yêu cầu của Bài 1 Bài 1: So sánh AB, BA: Cho các ma trận:  $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} B = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$

```
import numpy as np

# 1. Khởi tạo các ma trận A và B từ đề bài
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[4, 3],
              [2, 1]])

# 2. Thực hiện phép nhân ma trận
# Sử dụng toán tử @ (tương đương với np.matmul)
AB = A @ B
BA = B @ A

# 3. Hiển thị kết quả
print("Ma trận A:\n", A)
print("\nMa trận B:\n", B)
print("-" * 20)
print("Kết quả AB:\n", AB)
print("\nKết quả BA:\n", BA)
print("-" * 20)

# 4. So sánh AB và BA
if np.array_equal(AB, BA):
    print("Kết luận: AB = BA")
else:
    print("Kết luận: AB ≠ BA")
    print("Giải thích: Trong đại số tuyến tính, phép nhân ma trận không có tính chất giao hoán.")
```

```
Ma trận A:
[[1 2]
 [3 4]]

Ma trận B:
[[4 3]
 [2 1]]
-----
Kết quả AB:
[[ 8  5]
 [20 13]]

Kết quả BA:
[[13 20]
 [ 5  8]]
-----
Kết luận: AB ≠ BA
Giải thích: Trong đại số tuyến tính, phép nhân ma trận không có tính chất giao hoán.
```

Bài 2: Với  $a + c = d$  và  $2c = 3b$ , Chứng minh  $AB = BA$  Cho các ma trận:  $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} B = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$

Prompt: Tạo Python code để thực hiện yêu cầu của Bài 2: Bài 2: Với  $a + c = d$  và  $2c = 3b$ , Chứng minh  $AB = BA$  Cho các ma trận:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} B = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

### ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

```
import sympy as sp

# 1. Định nghĩa các biến ký hiệu (symbols)
a, b, c, d = sp.symbols('a b c d')

# 2. Thiết lập các ma trận A và B từ đề bài
A = sp.Matrix([[1, 2],
               [3, 4]])

B = sp.Matrix([[a, b],
               [c, d]])

# 3. Tính tích AB và BA
AB = A * B
BA = B * A

# 4. Tính hiệu của hai tích ma trận (Diff = AB - BA)
# Nếu AB = BA thì Diff phải bằng ma trận không (Zero Matrix)
diff = AB - BA

print("Ma trận hiệu (AB - BA) ban đầu:")
sp.pprint(diff)

# 5. Áp dụng các điều kiện từ đề bài:
# Điều kiện 1: a + c = d => d = a + c
# Điều kiện 2: 2c = 3b    => b = 2c/3
conditions = [
    (d, a + c),
    (b, sp.Rational(2, 3) * c)
]

# Thay thế các điều kiện vào ma trận hiệu
simplified_diff = diff.subs(conditions)

print("\nMa trận hiệu sau khi áp dụng điều kiện a+c=d và 2c=3b:")
sp.pprint(simplified_diff)

# 6. Kết luận
if simplified_diff == sp.zeros(2, 2):
    print("\nKết luận: AB - BA = 0 => AB = BA (Đã chứng minh xong).")
else:
    print("\nKết luận: AB không bằng BA với các điều kiện đã cho.")
```

Ma trận hiệu (AB - BA) ban đầu:

$$\begin{bmatrix} -3b + 2c & -2a - 3b + 2d \\ 3a + 3c - 3d & 3b - 2c \end{bmatrix}$$

Ma trận hiệu sau khi áp dụng điều kiện a+c=d và 2c=3b:

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Kết luận:  $AB - BA = 0 \Rightarrow AB = BA$  (Đã chứng minh xong).

Bài 3: Tìm a thoả mãn ma trận sau:

$$a \begin{bmatrix} 1 & 3 & 4 \\ 2 & 1 & -1 \end{bmatrix} + \begin{bmatrix} 4 & 3 & -6 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 12 & 6 \\ 6 & 4 & -2 \end{bmatrix}$$

Prompt: Viết Python code để giải bài 3:

Bài 3: Tìm a thoả mãn ma trận sau:

$$a \begin{bmatrix} 1 & 3 & 4 \\ 2 & 1 & -1 \end{bmatrix} + \begin{bmatrix} 4 & 3 & -6 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 12 & 6 \\ 6 & 4 & -2 \end{bmatrix}$$

## ▼ Suu tầm và biên soạn: Prof Happy AI, 2026

```
import sympy as sp

# 1. Khai báo biến ẩn a
a = sp.symbols('a')

# 2. Định nghĩa các ma trận từ đề bài
# Ma trận hệ số nhân với a
M1 = sp.Matrix([[1, 3, 4],
                [2, 1, -1]])

# Ma trận cộng thêm
M2 = sp.Matrix([[4, 3, -6],
                [0, 1, 1]])

# Ma trận kết quả (vế phải)
M_target = sp.Matrix([[7, 12, 6],
                      [6, 4, -2]])

# 3. Thiết lập phương trình ma trận: a*M1 + M2 = M_target
# Tương đương với việc tìm a sao cho: a*M1 + M2 - M_target = Ma trận không
equation_matrix = a * M1 + M2 - M_target

print("Ma trận phương trình (các phần tử chứa ẩn a):")
sp.pprint(equation_matrix)

# 4. Giải phương trình cho một phần tử bất kỳ (ví dụ hàng 0, cột 0)
# Ta giải: 1*a + 4 - 7 = 0
solution = sp.solve(equation_matrix[0, 0], a)

# 5. Kiểm tra lại nghiệm vừa tìm được có đúng với tất cả các phần tử khác không
a_val = solution[0]
is_correct = (a_val * M1 + M2) == M_target

# 6. In kết quả
print("-" * 30)
if is_correct:
    print(f"Giá trị a tìm được là: a = {a_val}")
    print("Kiểm tra lại: Nghiệm này hoàn toàn thỏa mãn toàn bộ ma trận.")
else:
    print("Không tìm được giá trị a duy nhất thỏa mãn tất cả các phần tử.")
```

Ma trận phương trình (các phần tử chứa ẩn a):

$$\begin{bmatrix} a - 3 & 3a - 9 & 4a - 12 \\ 2a - 6 & a - 3 & 3 - a \end{bmatrix}$$

-----

Giá trị a tìm được là: a = 3

Kiểm tra lại: Nghiệm này hoàn toàn thỏa mãn toàn bộ ma trận.

Bài 4: Ma trận nghịch đảo Cho ma trận  $A$  như sau:

$$A = \begin{pmatrix} 1 & 3 & 2 \\ 2 & 1 & 3 \\ 3 & 2 & 1 \end{pmatrix}$$

Prompt: Viết Python code để giải bài 4: Bài 4: Ma trận nghịch đảo Cho ma trận  $A$  như sau:

$$A = \begin{pmatrix} 1 & 3 & 2 \\ 2 & 1 & 3 \\ 3 & 2 & 1 \end{pmatrix}$$

## ▼ Suu tầm và biên soạn: Prof Happy AI, 2026

```
import sympy as sp

# 1. Khởi tạo ma trận A từ đề bài
A = sp.Matrix([[1, 3, 2],
               [2, 1, 3],
               [3, 2, 1]])

print("Ma trận A:")
sp.pprint(A)

# 2. Kiểm tra định thức (det) của ma trận
```

```
# Một ma trận có nghịch đảo khi và chỉ khi định thức khác 0
det_A = A.det()
print(f"\nĐịnh thức của A: det(A) = {det_A}")

if det_A != 0:
    # 3. Tính ma trận nghịch đảo
    A_inv = A.inv()

    print("\nMa trận nghịch đảo A^-1 là:")
    sp.pprint(A_inv)

    # 4. Kiểm tra lại: A * A_inv phải bằng ma trận đơn vị I
    print("\nKiểm tra lại (A * A^-1):")
    sp.pprint(A * A_inv)
else:
    print("\nMa trận A không có ma trận nghịch đảo vì det(A) = 0.")
```

Ma trận A:

$$\begin{bmatrix} 1 & 3 & 2 \\ 2 & 1 & 3 \\ 3 & 2 & 1 \end{bmatrix}$$

Định thức của A:  $\det(A) = 18$

Ma trận nghịch đảo  $A^{-1}$  là:

$$\begin{bmatrix} -5/18 & 1/18 & 7/18 \\ 7/18 & -5/18 & 1/18 \\ 1/18 & 7/18 & -5/18 \end{bmatrix}$$

Kiểm tra lại  $(A * A^{-1})$ :

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Bài 5: Tính định thức ma trận cấp 4 sau:

$$D = \begin{vmatrix} 2 & -3 & 4 & 1 \\ 4 & -2 & 3 & 2 \\ a & b & c & d \\ 3 & -1 & 4 & 3 \end{vmatrix}$$

Prompt: Viết Python code để giải bài 5:

Bài 5: Tính định thức ma trận cấp 4 sau:

$$D = \begin{vmatrix} 2 & -3 & 4 & 1 \\ 4 & -2 & 3 & 2 \\ a & b & c & d \\ 3 & -1 & 4 & 3 \end{vmatrix}$$

## ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

```
import sympy as sp

# 1. Khai báo các biến ký hiệu
a, b, c, d = sp.symbols('a b c d')

# 2. Định nghĩa ma trận D từ đề bài
# Lưu ý: Các hàng được nhập theo đúng thứ tự trong hình
D_matrix = sp.Matrix([
    [2, -3, 4, 1],
    [4, -2, 3, 2],
    [a, b, c, d],
    [3, -1, 4, 3]
])

print("Ma trận D cần tính định thức:")
sp.pprint(D_matrix)

# 3. Tính định thức
# Kết quả sẽ là một biểu thức phụ thuộc vào a, b, c, d
determinant_D = D_matrix.det()
```

```
# 4. Rút gọn biểu thức
simplified_result = sp.simplify(determinant_D)

print("\n" + "="*30)
print("Kết quả định thức D là:")
print(simplified_result)
print("="*30)

# Hiển thị dưới dạng toán học đẹp mắt (nếu chạy trên Jupyter/Colab)
# sp.init_printing()
# display(simplified_result)
```

Ma trận D cần tính định thức:

$$\begin{vmatrix} 2 & -3 & 4 & 1 \\ 4 & -2 & 3 & 2 \\ a & b & c & d \\ 3 & -1 & 4 & 3 \end{vmatrix}$$

=====

Kết quả định thức D là:

$$8*a + 15*b + 12*c - 19*d$$

=====

Bài 6: Tính định thức:

$$D = \begin{vmatrix} 1 & -2 & 1 & 4 & 10 \\ 1 & 3 & 2 & 5 & 3 \\ 0 & 5 & 3 & 7 & 9 \\ 0 & 0 & 2 & 3 & 7 \\ 0 & 0 & 0 & 3 & 15 \end{vmatrix}$$

Prompt: Viết Python code để giải bài 6 như sau: Bài 6: Tính định thức:

$$D = \begin{vmatrix} 1 & -2 & 1 & 4 & 10 \\ 1 & 3 & 2 & 5 & 3 \\ 0 & 5 & 3 & 7 & 9 \\ 0 & 0 & 2 & 3 & 7 \\ 0 & 0 & 0 & 3 & 15 \end{vmatrix}$$

## ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

```
import sympy as sp

# 1. Khởi tạo ma trận D dựa trên đề bài
D_matrix = sp.Matrix([
    [1, -2, 1, 4, 10],
    [1, 3, 2, 5, 3],
    [0, 5, 3, 7, 9],
    [0, 0, 2, 3, 7],
    [0, 0, 0, 3, 15]
])

print("Ma trận D (cấp 5):")
sp.pprint(D_matrix)

# 2. Tính định thức
# Thư viện sympy sẽ sử dụng các phép biến đổi hàng hoặc khai triển Laplace
det_D = D_matrix.det()

print("\n" + "="*25)
print(f"Giá trị định thức D = {det_D}")
print("="*25)

# 3. Giải thích thêm (tùy chọn)
# Kiểm tra xem ma trận có suy biến hay không
if det_D == 0:
    print("Kết luận: Ma trận D suy biến (singular matrix).")
else:
    print("Kết luận: Ma trận D không suy biến.")
```

Ma trận D (cấp 5):

$$\begin{vmatrix} 1 & -2 & 1 & 4 & 10 \\ 1 & 3 & 2 & 5 & 3 \\ 0 & 5 & 3 & 7 & 9 \\ 0 & 0 & 2 & 3 & 7 \\ 0 & 0 & 0 & 3 & 15 \end{vmatrix}$$

$$\begin{bmatrix} 0 & 5 & 3 & 7 & 9 \\ 0 & 0 & 2 & 3 & 7 \\ 0 & 0 & 0 & 3 & 15 \end{bmatrix}$$

=====

Giá trị định thức D = -180

=====

Kết luận: Ma trận D không suy biến.

Bài 7: Tìm hạng của ma trận  $A = \begin{pmatrix} 0 & 1 & 10 & 3 \\ 2 & 0 & 4 & -1 \\ 16 & 4 & 52 & 9 \\ 8 & -1 & 6 & -7 \end{pmatrix}$

Prompt: Viết Python code để giải bài 7:

Bài 7: Tìm hạng của ma trận  $A = \begin{pmatrix} 0 & 1 & 10 & 3 \\ 2 & 0 & 4 & -1 \\ 16 & 4 & 52 & 9 \\ 8 & -1 & 6 & -7 \end{pmatrix}$

## ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

```
import numpy as np
import sympy as sp

# Ma trận A từ đề bài
data = [[0, 1, 10, 3],
        [2, 0, 4, -1],
        [16, 4, 52, 9],
        [8, -1, 6, -7]]

# Cách 1: Sử dụng NumPy (Thường dùng trong tính toán số thực)
A_np = np.array(data)
rank_np = np.linalg.matrix_rank(A_np)

# Cách 2: Sử dụng SymPy (Cho kết quả chính xác tuyệt đối, phù hợp với bài tập toán)
A_sp = sp.Matrix(data)
rank_sp = A_sp.rank()

print(f"Ma trận A: \n{A_sp}")
print("-" * 20)
print(f"Hạng của ma trận A (tính bằng NumPy): {rank_np}")
print(f"Hạng của ma trận A (tính bằng SymPy): {rank_sp}")

# Hiển thị các bước đưa về dạng bậc thang (RREF) để hiểu rõ hơn
print("\nMa trận đưa về dạng bậc thang rút gọn (RREF):")
sp.pprint(A_sp.rref()[0])
```

Ma trận A:  
Matrix([[0, 1, 10, 3], [2, 0, 4, -1], [16, 4, 52, 9], [8, -1, 6, -7]])

Hạng của ma trận A (tính bằng NumPy): 3  
Hạng của ma trận A (tính bằng SymPy): 3

Ma trận đưa về dạng bậc thang rút gọn (RREF):

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 11/2 \\ 0 & 0 & 1 & -1/4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Bài 8: Biện luận hạng của ma trận sau:

$$A = \begin{pmatrix} -3 & 9 & 14 & 1 \\ 0 & 6 & 10 & 2 \\ 1 & 4 & 7 & 2 \\ 3 & \lambda & 1 & 2 \end{pmatrix}$$

Prompt viết python code giải bài 8:

Bài 8: Biện luận hạng của ma trận sau:

$$A = \begin{pmatrix} -3 & 9 & 14 & 1 \\ 0 & 6 & 10 & 2 \\ 1 & 4 & 7 & 2 \\ 3 & \lambda & 1 & 2 \end{pmatrix}$$

## ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

```
import sympy as sp

# 1. Khai báo biến ký hiệu cho tham số lambda
lam = sp.symbols('lambda')

# 2. Định nghĩa ma trận A từ đề bài
A = sp.Matrix([
    [-3, 9, 14, 1],
    [0, 6, 10, 2],
    [1, 4, 7, 2],
    [3, lam, 1, 2]
])

print("Ma trận A với tham số lambda:")
sp.pprint(A)

# 3. Đưa ma trận về dạng bậc thang (Gaussian Elimination)
# Chúng ta sử dụng phương pháp biến đổi sơ cấp để làm xuất hiện các số 0
# Cách nhanh nhất để biện luận là tính định thức nếu là ma trận vuông,
# hoặc đưa về dạng bậc thang.
rref_matrix, pivot_columns = A.rref()

print("\nMa trận dạng bậc thang rút gọn (RREF):")
sp.pprint(rref_matrix)

# 4. Tính định thức của ma trận A (vì đây là ma trận vuông 4x4)
# Hạng của ma trận sẽ thấp hơn 4 nếu định thức bằng 0
det_A = A.det()
print(f"\nĐịnh thức của ma trận A: det(A) = {det_A}")

# 5. Giải phương trình det(A) = 0 để tìm giá trị lambda đặc biệt
critical_lambdas = sp.solve(det_A, lam)

print("\n--- BIỆN LUẬN ---")
if not critical_lambdas:
    print(f"Hạng của ma trận luôn bằng {len(pivot_columns)} với mọi lambda.")
else:
    for val in critical_lambdas:
        A_sub = A.subs(lam, val)
        print(f"Nếu lambda = {val}:")
        print(f"=> Hạng của ma trận rank(A) = {A_sub.rank()}")

    print(f"Nếu lambda ≠ {critical_lambdas[0]}:")
    print(f"=> Hạng của ma trận rank(A) = 4")
```

Ma trận A với tham số lambda:

$$\begin{bmatrix} -3 & 9 & 14 & 1 \\ 0 & 6 & 10 & 2 \\ 1 & 4 & 7 & 2 \\ 3 & \lambda & 1 & 2 \end{bmatrix}$$

Ma trận dạng bậc thang rút gọn (RREF):

$$\begin{bmatrix} 1 & 0 & 0 & 3/5 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1/5 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Định thức của ma trận A:  $\det(A) = 0$

--- BIỆN LUẬN ---

Hạng của ma trận luôn bằng 3 với mọi lambda.

Bài 9: Giải hệ phương trình tuyến tính sau:

$$\begin{cases} 7x_1 + 2x_2 + 3x_3 = 15 \\ 5x_1 - 3x_2 + 2x_3 = 15 \\ 10x_1 - 11x_2 + 5x_3 = 36 \end{cases}$$

Prompt Viết Python code giải bài 9 sau đây:

Bài 9: Giải hệ phương trình tuyến tính sau:

$$\begin{cases} 7x_1 + 2x_2 + 3x_3 = 15 \\ 5x_1 - 3x_2 + 2x_3 = 15 \\ 10x_1 - 11x_2 + 5x_3 = 36 \end{cases}$$

## ✓ **Sưu tầm và biên soạn: Prof Happy AI, 2026**

```
import numpy as np
import sympy as sp

# Khai báo các hệ số của hệ phương trình
# Hàng 1: 7x1 + 2x2 + 3x3 = 15
# Hàng 2: 5x1 - 3x2 + 2x3 = 15
# Hàng 3: 10x1 - 11x2 + 5x3 = 36

# Ma trận hệ số A
A_data = [[7, 2, 3],
           [5, -3, 2],
           [10, -11, 5]]

# Ma trận kết quả B
B_data = [15, 15, 36]

# --- CÁCH 1: Dùng SymPy (Kết quả chính xác dạng phân số/số nguyên) ---
A_sp = sp.Matrix(A_data)
B_sp = sp.Matrix(B_data)

# Giải hệ phương trình A*X = B
solution_sp = A_sp.solve(B_sp)

print("Kết quả giải bằng SymPy (Chính xác):")
print(f"x1 = {solution_sp[0]}")
print(f"x2 = {solution_sp[1]}")
print(f"x3 = {solution_sp[2]}")

print("-" * 30)

# --- CÁCH 2: Dùng NumPy (Kết quả dạng số thực float) ---
A_np = np.array(A_data)
B_np = np.array(B_data)

try:
    solution_np = np.linalg.solve(A_np, B_np)
    print("Kết quả giải bằng NumPy (Số thực):")
    print(f"x1 = {solution_np[0]:.2f}")
    print(f"x2 = {solution_np[1]:.2f}")
    print(f"x3 = {solution_np[2]:.2f}")
except np.linalg.LinAlgError:
    print("Hệ phương trình vô nghiệm hoặc có vô số nghiệm.")
```

```
Kết quả giải bằng SymPy (Chính xác):
x1 = 2
x2 = -1
x3 = 1
-----
Kết quả giải bằng NumPy (Số thực):
x1 = 2.00
x2 = -1.00
x3 = 1.00
```

Bài 10: Tìm dạng chính tắc của dạng toàn phương sau:

$$f(x_1, x_2, x_3) = x_1^2 + x_2^2 + 3x_3^2 + 4x_1x_2 + 2x_1x_3 + 2x_2x_3$$

Prompt viết Python code để giải bài 10: Bài 10: Tìm dạng chính tắc của dạng toàn phương sau:

$$f(x_1, x_2, x_3) = x_1^2 + x_2^2 + 3x_3^2 + 4x_1x_2 + 2x_1x_3 + 2x_2x_3$$



```

import sympy as sp

# 1. Khai báo các biến số
x1, x2, x3 = sp.symbols('x1 x2 x3')

# 2. Định nghĩa hàm dạng toàn phương f từ đề bài
f = x1**2 + x2**2 + 3*x3**2 + 4*x1*x2 + 2*x1*x3 + 2*x2*x3

print(f"Dạng toàn phương ban đầu: f = {f}")

# 3. Xác định ma trận A của dạng toàn phương
# Hệ số của xi^2 nằm trên đường chéo chính
# Hệ số của xi*xj được chia đôi và nằm ở hai vị trí đối xứng
A = sp.Matrix([
    [1, 2, 1],
    [2, 1, 1],
    [1, 1, 3]
])

print("\nMa trận A của dạng toàn phương:")
sp.pprint(A)

# 4. Tìm các trị riêng (Eigenvalues)
# Các trị riêng chính là các hệ số của dạng chính tắc trong phép biến đổi trực giao
eigenvals = A.eigenvals()

print("\nCác trị riêng tìm được:")
for val, mult in eigenvals.items():
    print(f"Trị riêng λ = {val} (bội {mult})")

# 5. Xuất dạng chính tắc
print("\n" + "="*30)
print("DẠNG CHÍNH TẮC (Phương pháp biến đổi trực giao):")
canonical_form = 0
y = sp.symbols('y1 y2 y3')
for i, (val, mult) in enumerate(eigenvals.items()):
    # Lưu ý: nếu trị riêng có bội > 1, cần cộng dồn cho các biến y tương ứng
    for m in range(mult):
        canonical_form += val * y[i+m]**2

print(f"f = {canonical_form}")
print("="*30)

# 6. Kiểm tra tính xác định
if all(v > 0 for v in eigenvals.keys()):
    print("Kết luận: Dạng toàn phương xác định dương.")
elif all(v < 0 for v in eigenvals.keys()):
    print("Kết luận: Dạng toàn phương xác định âm.")
else:
    print("Kết luận: Dạng toàn phương không xác định dấu.")

```

Dạng toàn phương ban đầu:  $f = x1^2 + 4*x1*x2 + 2*x1*x3 + x2^2 + 2*x2*x3 + 3*x3^2$

Ma trận A của dạng toàn phương:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Các trị riêng tìm được:

Trị riêng  $\lambda = -1$  (bội 1)

Trị riêng  $\lambda = 3 - \sqrt{2}$  (bội 1)

Trị riêng  $\lambda = \sqrt{2} + 3$  (bội 1)

=====

DẠNG CHÍNH TẮC (Phương pháp biến đổi trực giao):

$f = -y1^2 + y2^2*(3 - \sqrt{2}) + y3^2*(\sqrt{2} + 3)$

=====

Kết luận: Dạng toàn phương không xác định dấu.